Brigham Young University – Idaho
Department of Electrical & Computer Engineering
ECEN 340 – Digital System Design

FPGA-Based Fan Condition Monitoring System

System Architecture, Data Acquisition, and Classification

Team: Matt Eyre, Nathan Baldauf, Tanner Norton
Instructor: Brother Randall Jack
Date: July 20, 2026

Contents

1 Problem Statement

For decades, mechanical systems have relied heavily on rotational motion, where innovations like precision-machined bearings have vastly improved operating efficiency. A significant drawback of these precise mechanisms, however, is their extreme sensitivity to dynamic unbalance, especially at high rotational speeds. Left unaddressed, unbalanced rotation causes severe vibration and premature component failure. To enable proactive preventative maintenance, this project develops a real-time monitoring system designed to capture, process, and accurately diagnose unbalanced vibrational signatures in rotational hardware.

2 Objectives

To test our concept and help extend the life of rotational components, we wanted to analyze the vibrational patterns of a fan in a balanced state and an intentionally unbalanced state. To do so, we would communicate with an ADXL345, 3-axis accelerometer attached to the fan. An FPGA will read out the data from the accelerometer at a slower clock rate and write into the first FIFO. At 100 MHz, the FPGA will read out of the FIFO and perform a windowing operation at 100 MHz, writing that data into a second FIFO. Controlled by the SPI clock of the MCU, data will be read out of the second FIFO into the MCU, where a Fast-Fourier Transform will be performed. Frequency behavior of the vibrations of the fan will be analyzed by a neural network operating on the MCU, which will identify whether the fan is off, balanced, or unbalanced.

3 Materials

- Fan
- ADXL345 3-Axis Digital Accelerometer
- Digilent Basys 3 Artix-7 FPGA
- NUCLEO-L476RG
- LCD1602 Module
- Jumper wires
- Logic analyzer

4 System Architecture

4.1 accel_FSM module

Essential to the success of the rest of the project was learning how to communicate with the accelerometer. Key to this was careful study of the ADXL345 datasheet from Analog Devices. In order to configure the accelerometer, we needed to write data to it. Figure 1 shows the timing diagram for a sample write command. Once chip select goes low (indicating

the start of a read or write command) the following clock cycle marks the first bit written to the accelerometer. A new bit is read on each clock. The first bit being low indicates a write command. Following is a multi-bit true or false, which allows data to be written to the accelerometer continuously, without sending new address information. Next, there are 6 bits of address information that determines where data to the accelerometer should be written to, followed by 8 bits of configuration data.

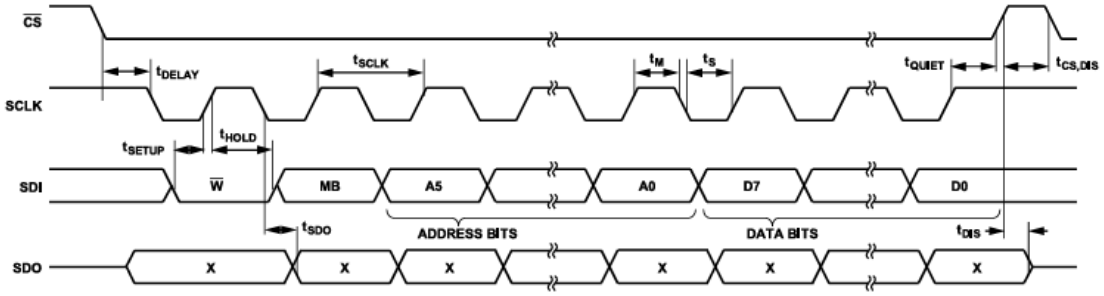

Figure 37. SPI 4-Wire Write

Figure 1: SPI 4-Wire Write (from ADXL345 datasheet)

Figure 2 shows the process of a read command. After sending the R/W bit high, multi-bit high, and the reading address, the accelerometer will continuously read out data until CS goes high again. We wanted 16 bit samples from the accelerometer, so we allowed it to read out 16 bits of data before setting CS high again.

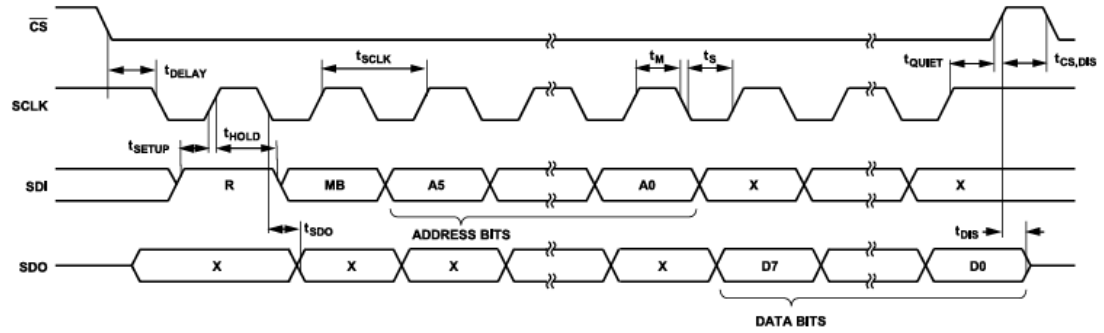

Figure 38. SPI 4-Wire Read

Figure 2: SPI 4-Wire Read (from ADXL345 datasheet)

We determined that it would be easiest to keep track of these states by using a state machine. As the state machine moves from state to state, the required operations happen automatically. We had an idle state, configuration state, and read state. Upon pressing the reset button on the FPGA (btnC in our case) config data was written to the accelerometer in one giant string of bits. We assembled this with several parameters, as seen in Listing 1.

```
// Bits 7 & 6 = FIFO mode
// Bit 0 sets watermark level to 1
parameter FIFO_CONFIG = {W, MB_F, FIFO_ADDR,
                         8'b01000001}, // 16 clock cycles
          // Bit 15 = DATA_READY
          // Bit 8 = Overrun
          // Bit 7 = DATA_READY on interrupt line 1
          // Bit 0 = Overrun on interrupt line 2
          INTERR_CONFIG = {W, MB_T, INTERR_ADDR,
                           16'b1000_0001_0000_0001},
                           // 24 clock cycles
          SET_800Hz_SAMPLE = {W, MB_F, SAMPLE_ADDR,
                              8'b1101}, // 16 clock cycles
          SET_MEASURE_BIT = {W, MB_F, MEASURE_BIT_ADDR,
                             8'h08}, // 16 clock cycles
          // Bit 3 = FULL_RES bit
          // Bits 1 & 0 = Range bits
          SET_DATA_FORMAT = {W, MB_F, DATA_FORMAT_ADDR,
                             8'b0000_0101}, // 16 clock cycles
          READ_CMD = {R, MB_T, READ_ADDR}, // 8 bits starting a
             read
          WRITE_CONFIG_DATA = {NS, NS, FIFO_CONFIG, NS, NS,
              INTERR_CONFIG, NS, NS, SET_800Hz_SAMPLE, NS, NS,
              SET_DATA_FORMAT, NS, NS, SET_MEASURE_BIT};
```

Listing 1: Configuration parameter definitions

Following configuration, the finite state machine enters an idle state, waiting for a DATA_READY signal from the accelerometer. Upon DATA_READY going high, the FSM enters a read state, sending the bits in the READ_CMD parameter as seen above, and receiving 16 bits data before entering an idle state again. Once configured, the FSM only goes back and forth between reading and idle until it's reset again. The FSM module takes the serial data coming in via the SPI connection, parallelizes it, and outputs it to the accel_FIFO module as 16-bits parallel. The elaborated design of the accel_FSM module can be seen in Figure 3.

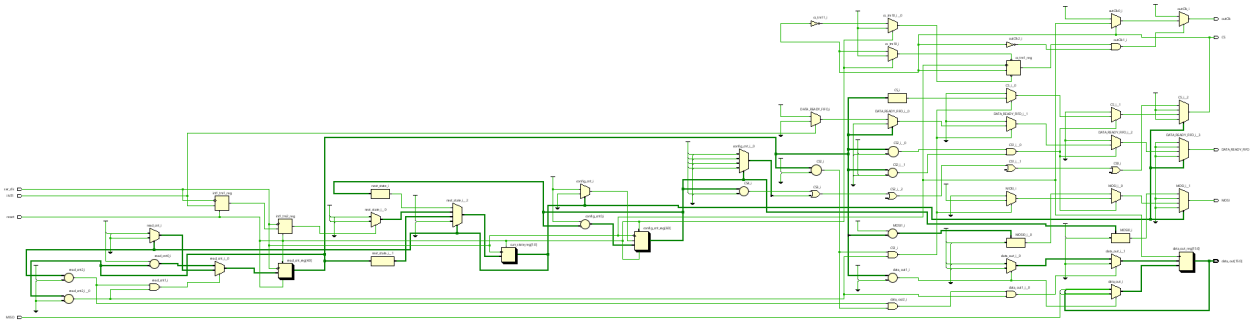

Figure 3: Elaborated design of accel_FSM module

To get this module functioning properly was very difficult. Initially, we saw what we were hoping to see in a test bench. However, we learned that proper behavior in a test bench does not always equate to proper behavior with real hardware. We found, using a logic analyzer and significant time troubleshooting, that we had multiple issues with timing violations. We learned the importance of careful study of timing requirements the hard way. Upon correcting these timing violations, the accel_FSM module smoothly and efficiently sent config data to and received data from the accelerometer. Listing 2 contains Verilog code for the FSM output logic in a read state.

```
READ: begin
    DATA_READY_FIFO = 1'b0;
    MOSI = 1'b1;
    CS = 1'b0;

    // Bits 24-17 are command and address data
    if (read_cnt <= 24 && read_cnt > 16) begin
        MOSI = READ_CMD[read_cnt - 17];
        CS = 1'b0;
    // Bits 16-1 are accelerometer data
    end else if (read_cnt <= 16 && read_cnt > 0) begin
        MOSI = 1'b0;
        CS = 1'b0;
    // Bit 0 set data ready flag for FIFO
    end else if (read_cnt == 0) begin
        if (!INT1) begin
            DATA_READY_FIFO = 1;
        end else begin
            DATA_READY_FIFO = 0;
        end
        CS = 1;
    end
end
end
```

Listing 2: FSM read state output logic

4.2 accel_FIFO module

Data is read out from the accelerometer into the FPGA on a 1 MHz SPI clock. Because the following module is operating on a 100 MHz clock we had to use a FIFO to handle crossing clock domains while avoiding metastability. To do so, we used the LogiCORE IP FIFO Generator that ships with Vivado. The accel_FIFO module handles gating the write and read enables of the FIFO based on flag signals the FIFO generates, such as empty and full signals, as well as write and read reset busy signals. It's crucial that no data is written to or read from the FIFO while it's resetting, as doing so will essentially brick the FIFO. The FIFO IP is very particular about timing. For example, for an asynchronous reset of the FIFO (which is required upon initialization) the reset must be asserted for at least 3

clock cycles of the slowest clock in the FIFO. This is due to the fact that asynchronous reset signals are synchronized internally to prevent metastability. This required delay didn't cause issues with the performance of the FIFO on real hardware because a human pressing a reset button lasts hundreds of clock cycles on the 1 Mhz clock. However, in the simulation, we had to ensure our reset button was asserted long enough to successfully start a reset. The testbench simulation can be seen below in figures 5 and 6.

To go into detail about what's happening in the waveform, it can be seen that the FIFO initializes with both the empty and full signals asserted high. This is to prevent reading or writing from occurring before it's been successfully reset. The rd_rst_busy signal deasserts significantly sooner than the wr_rst_busy signal because the reading clock is 100x faster than the writing clock. When the wr_rst_busy signal goes low, the WE signal goes high and the testbench begins writing to the FIFO. The testbench has been configured to write numbers 1 through 1024 in order to the FIFO.

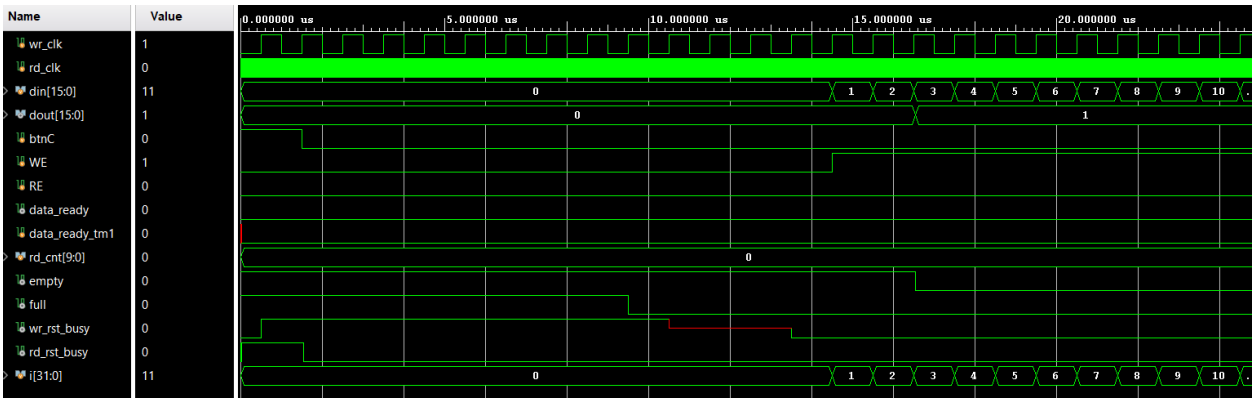

Figure 4: accel_FIFO testbench writing simulation

Figure 6 shows the simulation at the point where data is read out of the FIFO. When all 1024 samples have been written into the FIFO, the data_ready flag goes high. The test bench is configured to set the read enable signal high when the data_ready signal is high (simulating what the windowing function following the FIFO would do. Data is read out of the FIFO, in order, at the rate of the reading clock (100 MHz).

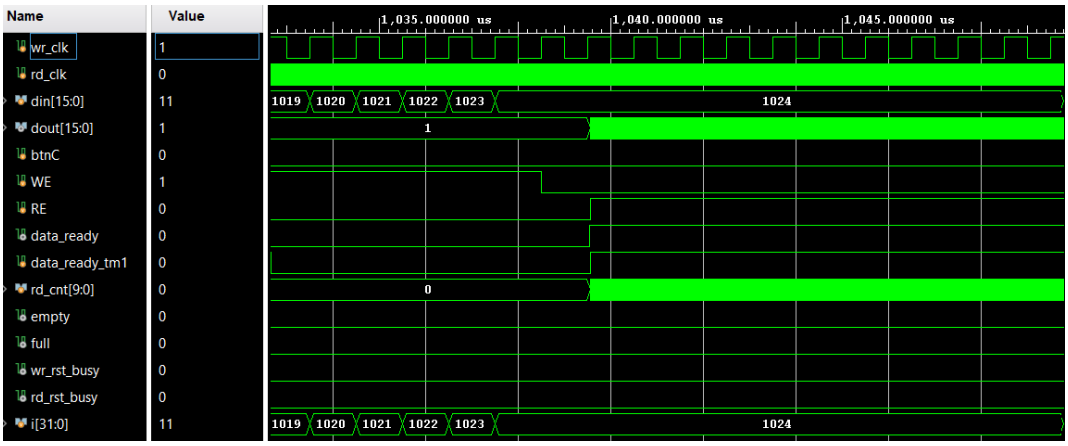

Figure 5: accel_FIFO testbench reading simulation

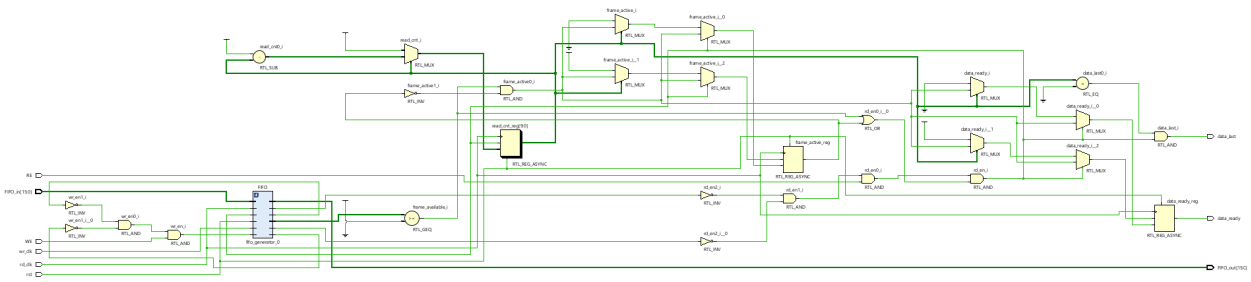

Figure 6: accel_FIFO schematic

4.3 Windowing Module

The windowing module applies a pre-defined window function to the incoming 16-bit data stream before it is processed by the FFT. It operates on a configurable FFT size, defaulting to 1024 samples.

The module interfaces directly with the preceding FIFO and the subsequent FFT module. It utilizes a Block RAM (BRAM) instantiated as window_rom to store the window coefficients (see Listing 3). An internal address counter, which increments upon valid data arrival (DA_in) and when the FFT is ready (FFT_ready), addresses this ROM (see Listing 5).

To maintain data alignment through the pipeline, the module delays the input data and control signals (see Listing 6). The core operation consists of a signed multiplication between the 16-bit delayed input data and the 16-bit unsigned window coefficient fetched from the ROM. The 32-bit multiplication result is then truncated and packed into the upper 16 bits of the 32-bit output data bus (data_out), which is forwarded to the FFT along with the pipelined control signals (see Listing 4). Stall protection is implemented by gating internal register updates with the FFT_ready signal.

```
blk_mem_gen_0 window_rom (
    .clka(clk),
    .ena(FFT_ready),
    .addra(addr_counter),
    .douta(window_coeff)
);

wire [32:0] mult_result32;
assign mult_result32 = $signed(data_in_delayed)
                     * $signed({1'b0, window_coeff});
assign mult_result = mult_result32[31:0];
```

Listing 3: BRAM instantiation and multiplication

```
always @(posedge clk) begin
    if (FFT_ready) begin
        DA_out <= DA_in_delayed;
        if (DA_in_delayed) begin
            data_out <= {16'b0, mult_result[31:16]};
            data_last_pipeline <= data_last_delay;
        end else begin
            data_last_pipeline <= 1'b0;
        end
    end
end
```

Listing 4: Output pipeline stage

```
always @(posedge clk or posedge rst) begin
    if (rst) begin
        addr_counter <= 0;
    end else if (DA_in && FFT_ready) begin
        if ((addr_counter == FFT_SIZE - 1) |
            (data_last == 1)) begin
            addr_counter <= 0;
        end else begin
            addr_counter <= addr_counter + 1;
        end
    end
end
```

Listing 5: Address counter logic

```
always @(posedge clk) begin
    if (FFT_ready) begin // Stall protection
        if (DA_in) begin
            data_in_delayed <= data_in;
            data_last_delay <= data_last;
        end
        DA_in_delayed <= DA_in;
    end
end
```

Listing 6: Input delay logic

The functionality of the windowing module has been verified through simulation. The testbench waveform demonstrates the correct application of the window function, showing the characteristic envelope applied to the output data across the full 1024-sample frame.

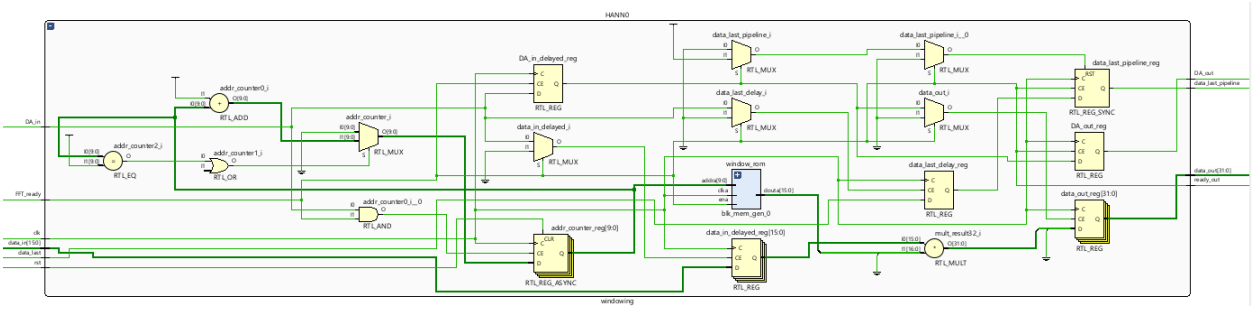

Figure 7: Hamming Window

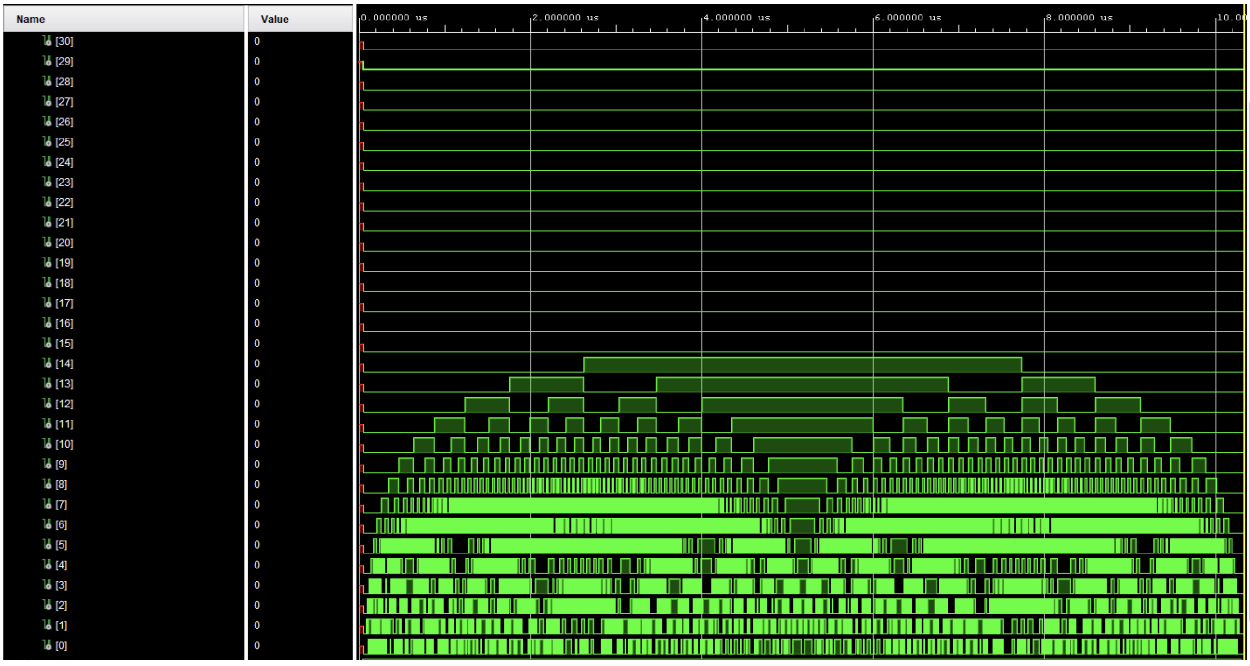

Figure 8: Windowing module testbench waveform across the 1024-sample frame

4.4 FIFO2

After the FFT is computed in the 100MHz domain, we need to shift the data into a 1MHz domain to shift the data out via SPI communication to the Microprocessor. This clock domain crossing (CDC) requires a FIFO to shift the data from one clock domain to another while preventing metastability. This is done by having a handshaking protocol with independent read and write clocks. These clocks are continuously running (per the FIFO IP specs, as triggering a reset while one or both clocks are stopped can put the FIFO in an invalid state) and as long as the FIFO is not full (FIFO_full flag is deasserted) a write operation occurs on each rising clock edge when wr_en is asserted.

When data is ready to be read (valid is high) rd_en can be asserted and then a new value will be presented on the dout bus, allowing the data to cross the clock domain while preventing metastability.

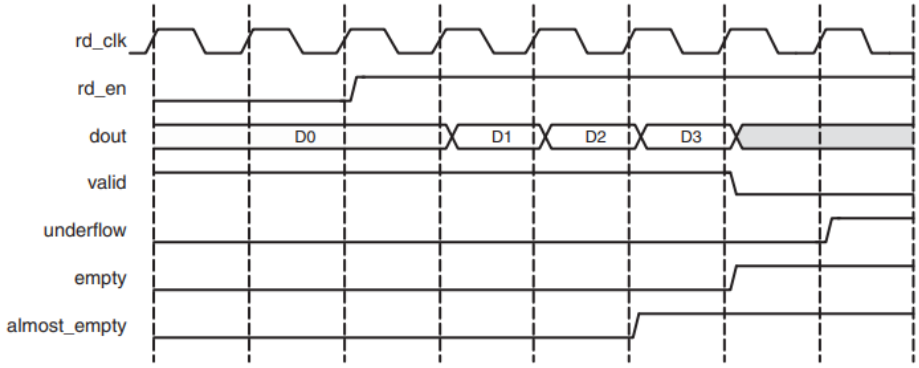

Figure 9: Independent Clock Read Operation (source: Xilinx IP datasheet)

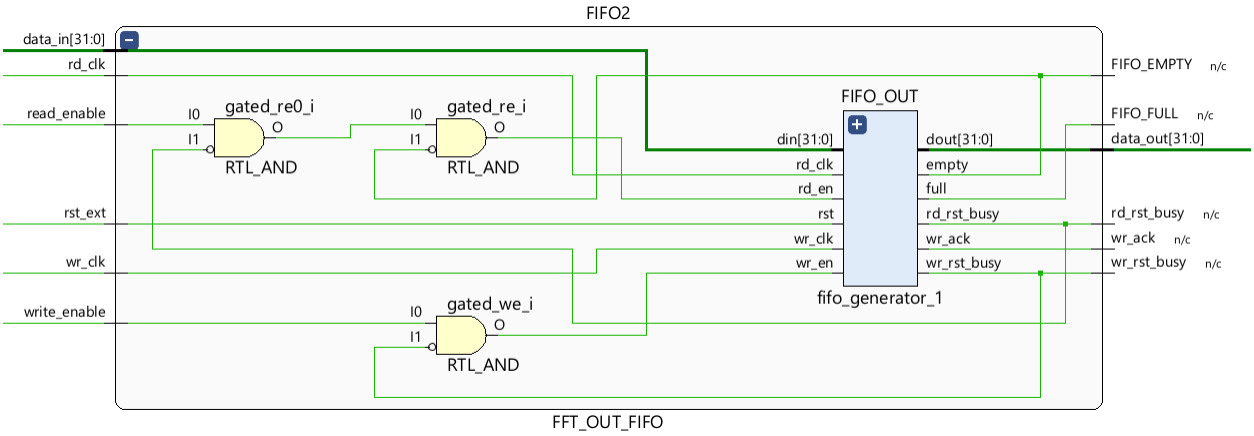

Figure 10: FFT_OUT_FIFO elaborated design

4.5 DROP_512/COUNT_1024

Different architectures (as discussed in the Obstacles and Alternative Architectures section) required different window frame sizes to be passed to the MCU over SPI. Because the MCU

expects an interrupt when the entire frame is ready to be sent, these modules count to 1024 as the samples arrive and pull a flag high when the frame is ready. DROP_512 holds wr_en low for the first 512 samples so they are never written into the FIFO, whereas COUNT_1024 still pulls the flag high but writes all 1024 samples to the FIFO

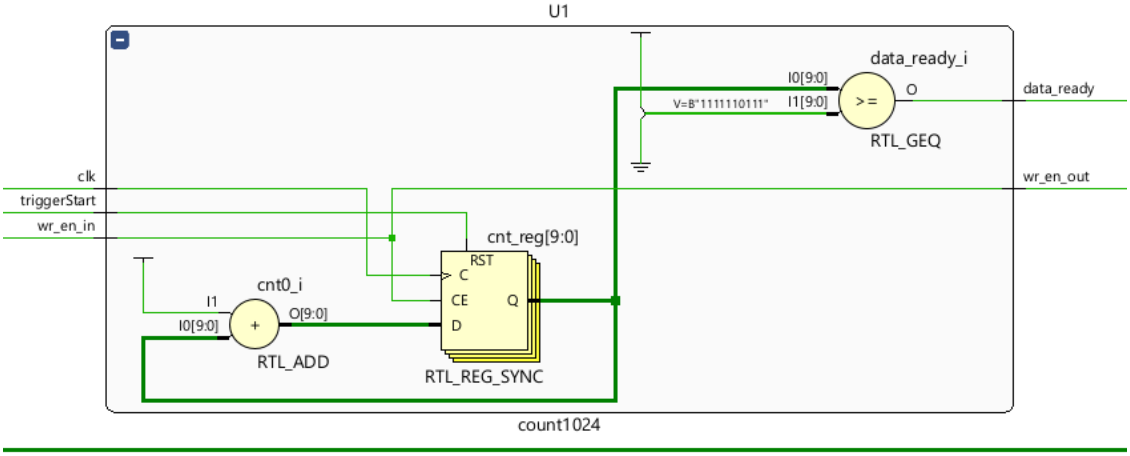

Figure 11: count1024 elaborated design

4.6 SPI SLAVE/MCU SPI MASTER

Because the MCU is responsible for pulling the data from the FPGA using an interrupt, the clock lines as well as chip select are generated by the MCU. A 1MHz clk from the MCU is divided by 32 (to give the SPI clk, which is 32 times faster, time to shift out every bit of the 32-bit data but) to become the read clk. This means that we are required to reset the read clk every time CS is pulled low to guarantee that the clk phase difference is within the system tolerable difference. The rd_en signal to the FIFO is directly tied to CS from the MCU.

The MCU collects 1024 packets via a SPI transaction and stores those directly into an array memory via a DMA (Direct Memory Access) operation. When the transaction is completed, the callback function sets a flag high to begin processing on the data.

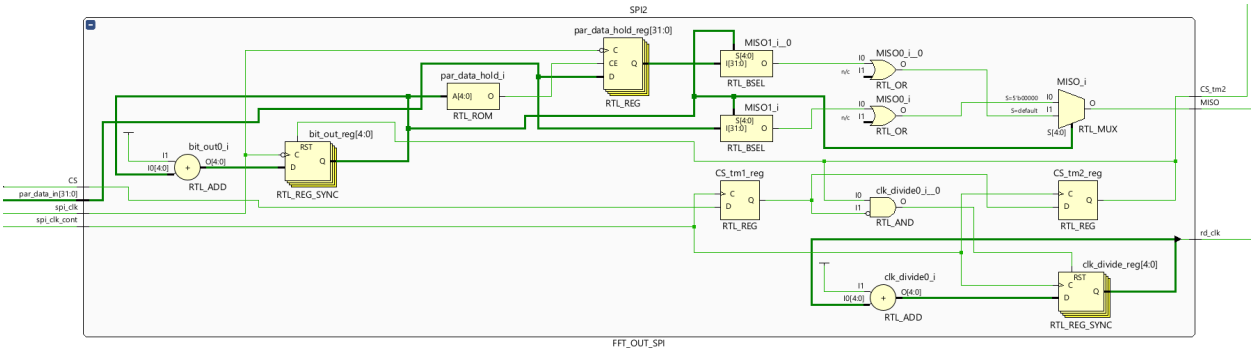

Figure 12: SPI slave elaborated design

```
always @(negedge spi_clk)
begin
    if (!CS_tm2) bit_out <= bit_out + 1;
    else bit_out <= 0;
    if (bit_out == 0) par_data_hold <= par_data_in;
    else par_data_hold <= par_data_hold;
end

// Increment our clock divide
always @(posedge spi_clk_cont)
begin
    if (CS_tm2 & ~CS_tm1) clk_divide <= 0;
    else clk_divide <= clk_divide + 1;
end

always @(posedge spi_clk_cont)
begin
    CS_tm1 <= CS;
    CS_tm2 <= CS_tm1;
end
```

Listing 7: CS synchronization and read-clock reset

```
assign rd_clk = clk_divide[4]; // rd_clk = spi_clk / 32
// While shifting from par_data_in to par_data_hold, use
// par_hold_data. Otherwise, use the saved data.
assign MISO = (bit_out == 0) ? par_data_in[bit_out] | CS
                             : par_data_hold[bit_out] | CS;
// On posedge of spi_clk (if CS), increment bit_out. If on
// the first bit (bit_out == 0), shift in new par_data.
```

Listing 8: Read-clock and MISO assignment

4.7 NN and Preprocessing

As the output of the FFT is a complex number in rectangular coordinates (representing magnitude and phase in polar coordinates), we need to extract the magnitude from the data (phase is irrelevant for our use case). This is done in the microcontroller for a complex number $\alpha + j\beta$ by performing $\sqrt{\alpha^2 + \beta^2}$ and saving the result into the input of the neural network.

Due to limited amount of ram available, a fully connected Neural Network was not possible with our input stages that would perform adequately. To remedy this, we binned the frequencies in 64 different bins, effectively shrinking our inputs from 512 down to 64. However, because of natural frequency variance of the fan this actually had a net positive result on the output, as small shifts in frequency had no effect on NN output.

The model is very effective at identifying when the fan is unbalanced due to the distinctive

frequency pattern, however due to the nominal difference of 0 between the powered down fan and the balanced fan the confidence of the model when differentiating them is far lower.

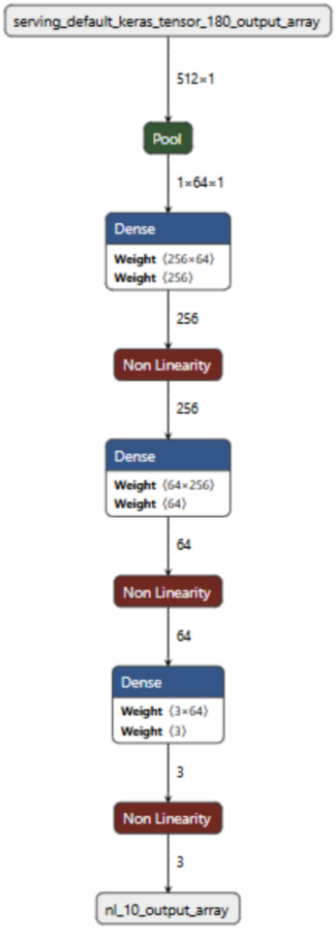

Figure 13: CNN Architecture

4.8 UART

In order to generate a large amount of training data for the Neural Network, the entire frame of data from the FPGA (after the MCU has taken the magnitude) is passed onto a laptop via UART, where a python script is monitoring the COM port. This UART transaction is also done via DMA as to not block the MCU from executing other functions.

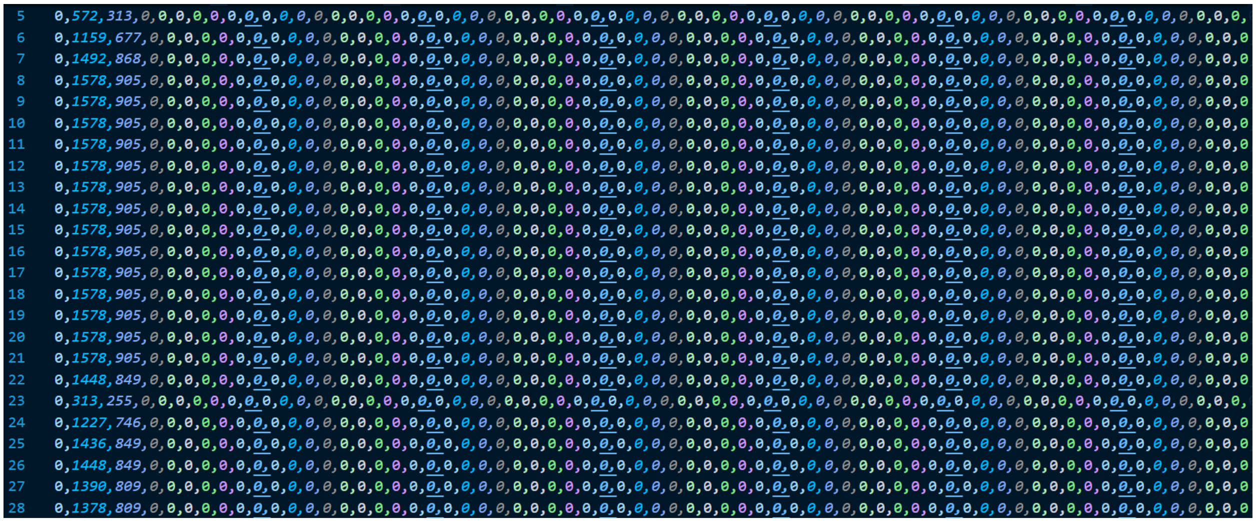

Figure 14: ML training data collected over UART

4.9 I2C

In order to display the results of the inference, a simple I2C display is used. A snprintf() function is used to concatenate the float of the inference confidence into the output of the inference result.

```
CharLCD_Clear();
switch (status) {
case 0:
    CharLCD_Set_Cursor(0,0); // Set cursor to row 0, column 0
    CharLCD_Write_String("Fan OFF");
    CharLCD_Set_Cursor(1,0); // Set cursor to row 1, column 0
    CharLCD_Write_String(confidence);
    break;
case 1:
    CharLCD_Set_Cursor(0,0); // Set cursor to row 0, column 0
    CharLCD_Write_String("Fan BALANCED");
    CharLCD_Set_Cursor(1,0); // Set cursor to row 1, column 0
    CharLCD_Write_String(confidence);
    break;
case 2:
    CharLCD_Set_Cursor(0,0); // Set cursor to row 0, column 0
    CharLCD_Write_String("Fan UNBALANCED");
    CharLCD_Set_Cursor(1,0); // Set cursor to row 1, column 0
    CharLCD_Write_String(confidence);
    break;
default:
    CharLCD_Set_Cursor(0,0); // Set cursor to row 0, column 0
    CharLCD_Write_String("ERROR!!");
    CharLCD_Set_Cursor(1,0); // Set cursor to row 1, column 0
    CharLCD_Write_String(confidence);
    break;
}
```

Listing 9: I2C display outputs

5 Obstacles and Alternative Architectures

We had 3 major issues throughout the course of the project that required extensive debugging, some of which prompted architecture changes. The first major issue was 2 simultaneous timing violations between the FPGA and accelerometer. One issue was that the accelerometer required 5ns between CS falling and the first falling edge of the SPI clk, however the SPI clk had a nominal delay of 0ns from the fall of CS. This caused the accelerometer to not register read commands, giving garbage data on the MISO line and not pulling the data available line low. To remedy this we built in a nominal delay of 1, as well as verifying that the data available line had registered the read. If it had not registered the read (meaning that the MISO line had not displayed real data) the data was discarded and another read was initialized. The second timing violation that was resolved was that the data available signal was not synchronized with the SPI FSM, meaning that the FSM could be put in an invalid state due to timing violations. To resolve this, the data available line was put through a 2-stage FIFO synchronizer, and all timing violations from the data available line ceased.

The second major issue was that 2 write enable logic lines had their polarity flipped, meaning that we were not writing when we expected to. This issues occurred in the second FIFO and to the accelerometer. To the accelerometer, when attempting to write configuration data due to flipped logic we were sending read commands, not configuring the accelerometer. With the FIFO, the wr_en logic was reversed so the system was being written to all the time, and when we wanted to write was the exact moment when writing would no longer be available.

The final major issue prompted a redesign as part of the troubleshooting process. The entire system seemed to be functional, however we would receive garbage data on the output. We assumed that there was some AXI stream protocol bug, and decided to try using the FFT in the microcontroller to see if that solved the issue. When the issue persisted, we knew that the problem was with the data pipeline before the FFT and found that while the data we were reading from the accelerometer was MSB first, it was least significant byte first. If the information we were trying to read could be represented by 0xAB, our FSM would read 0xBA. After resolving this, we continued using the FFT inside the MCU (as to not have to rewrite the MCU firmware so close to the deadline) however both versions are currently functional.

6 Results

The FPGA-Based Fan Condition Monitoring System successfully captured and processed vibrational data to classify the operational state of the fan. By interfacing the ADXL345 accelerometer with the Basys 3 FPGA, real-time acceleration data was reliably sampled and synchronized across clock domains using the custom FIFO and windowing modules.

The neural network, operating on the NUCLEO-L476RG microcontroller, effectively differentiated between the three target states: Off, Balanced, and Unbalanced. The system demonstrated high confidence when identifying the unbalanced state due to its distinct frequency signature. While differentiating between the powered-down and balanced states proved more challenging due to their similar nominal vibration profiles, the frequency binning preprocessing step successfully mitigated natural frequency variances, resulting in a robust classification model.

The final output was reliably displayed on the I2C LCD, confirming the end-to-end functionality of the data acquisition, processing, and classification pipeline.

7 Conclusion

This project was a wonderful learning experience! We successfully, and through much trial, created a device that would communicate with an accelerometer, process the time domain signals in an FPGA, output the signals to an MCU, perform an FFT, and analyze the frequency domain signals with a neural network. We learned how to apply FIFOs to cross clock domains. We also learned and applied important troubleshooting techniques when diagnosing errors (for there were many).

I'm grateful that we started on this project early and gave ourselves time to figure things

out. As Tom Cargill said "The first 90 percent of the code accounts for the first 90 percent of the development time. The remaining 10 percent of the code accounts for the other 90 percent of the development time." This project ended up being much more involved than we had anticipated, but I also learned so much in the process. Prior to this, my experience debugging on FPGA projects was very limited. I learned how valuable tools like test benches and also logic analyzers are. I also learned the importance of paying close attention to timing requirements. I'm used to using MCUs that operate more slowly and tend to be a little more forgiving. Because FPGAs work at blazing fast speeds, noting every single timing requirement is critical!